

CAHIER DES CHARGES TECHNIQUE

Application Mobile E-Commerce

AYANA HAIR

Projet	Application Mobile Ayana Hair
Type	Application Flutter + Backend Node.js + MySQL
Plateforme	Android, iOS, Web (Flutter multi-plateforme)
Version	1.0.0
Date	Mai 2026

SOMMAIRE

1. Présentation du projet
2. Architecture technique
3. Fonctionnalités côté Client
4. Fonctionnalités côté Administrateur
5. Base de données et diagramme de classes
6. API REST (pour les routes)
7. Sécurité
8. Structure des fichiers
9. Technologies utilisées
10. Livrables
11. Galerie (captures d'écran des interfaces de l'application sur émulateur Android Google Pixel 7)

1. Présentation du projet

1.1 Contexte

Ayana Hair est une boutique spécialisée dans les produits capillaires de qualité supérieure destinés aux femmes africaines. Face à la demande croissante, la boutique a souhaité se doter d'une application mobile complète permettant à ses clientes de commander en ligne et à son équipe de gérer les opérations efficacement.

1.2 Objectifs

- Offrir une expérience d'achat en ligne fluide et élégante
- Permettre la gestion complète du catalogue, des stocks et des commandes
- Notifier automatiquement les clients des mises à jour de leurs commandes
- Fournir un tableau de bord administrateur complet et connecté à la base de données

1.3 Périmètre

L'application couvre les fonctionnalités suivantes :

- Inscription et connexion sécurisée (JWT)
- Catalogue produits avec filtres par catégorie
- Panier persistant sauvegardé en base de données
- Tunnel de commande complet (livraison + paiement + récapitulatif)
- Historique et suivi des commandes
- Système de notifications automatiques
- Gestion du profil utilisateur
- Dashboard administrateur (stats, commandes, stocks)

2. Architecture technique

2.1 Architecture globale

L'application suit une architecture Client-Serveur classique à 3 couches :

Couche	Technologie
Frontend (Client)	Flutter (Dart) / Multi-plateforme Android/iOS/Web
Backend (Serveur)	Node.js + Express.js / API REST
Base de données	MySQL 8.x / Données persistantes
Authentification	JWT (JSON Web Token) / Expiration 24h
Communication	HTTP/HTTPS , JSON

2.2 Flux de données

Flutter → HTTP Request → Node.js API → MySQL → JSON Response → Flutter

2.3 Gestion de l'état

La gestion de l'état côté Flutter est assurée par le package Provider (ChangeNotifier). Le `PanierManager` est le principal gestionnaire d'état, synchronisant le panier entre l'interface et la base de données en temps réel.

3. Fonctionnalités côté Client

3.1 Authentification

Fonctionnalité	Description
Inscription	Formulaire avec nom, email, mot de passe (hashé bcrypt), type de cheveux
Connexion	Email + mot de passe, génération token JWT 24h
Déconnexion	Suppression du token local (SharedPreferences)
Persistance	Token et infos utilisateur stockés localement
Rôles	Deux rôles : client (accès app) et admin (accès dashboard)

3.2 Écran d'accueil (Home)

- Bannière avec image et bouton DÉCOUVRIR
- Section services : Livraison rapide, Produits authentiques, Support 24/7
- Grille de 6 produits phares avec images locales
- Icône panier avec badge affichant le nombre d'articles
- Barre de navigation inférieure : Accueil / Boutique / Profil
- Footer avec coordonnées de la boutique

3.3 Boutique

- Chargement dynamique des produits depuis l'API
- Filtres par catégorie : Tous, Huiles, Shampoings, Soins
- Grille 2 colonnes avec image, nom, prix, badge catégorie
- Bouton AJOUTER AU PANIER avec feedback visuel (SnackBar doré de confirmation de l'ajout d'un produit au panier)
- Badge panier en temps réel dans l'AppBar

3.4 Panier

- Liste des articles avec image, nom, prix unitaire
- Bouton supprimer par article (icône poubelle)
- Bouton Vider le panier (avec confirmation)
- Calcul automatique du sous-total et total
- Livraison gratuite affichée
- Persistance BDD : le panier est sauvegardé même après fermeture
- Synchronisation au login : chargement du panier depuis la BDD

3.5 Tunnel de commande (Checkout)

3 étapes avec navigation avant/arrière :

Étape 1 : Livraison

- Formulaire validé : Nom complet, Téléphone, Adresse de livraison
- Validation obligatoire avant de passer à l'étape suivante
- Info livraison gratuite en 24-48h

Étape 2 : Mode de paiement

- Orange Money (orange)
- MTN Mobile Money (jaune)
- Wave (bleu)
- Paiement à la livraison (doré)
- Sélection visuelle avec radio bouton animé

Étape 3 : Récapitulatif

- Résumé livraison, paiement et articles
- Total final affiché
- Bouton CONFIRMER LA COMMANDE → sauvegarde en BDD + vide le panier
- Dialog de succès avec retour automatique à l'accueil

3.6 Historique des commandes

- Liste de toutes les commandes de l'utilisateur connecté
- Affichage : numéro commande, date, heure, statut coloré, total
- Tap sur une commande affiche l'écran de détails
- Rafraîchissement par glissement (pull-to-refresh)

3.7 Détails d'une commande

- Statut avec couleur et icône (En attente / Confirmée / En livraison / Livrée / Annulée)
- Timeline visuelle : Commande passée → Confirmée → En livraison → Livrée
- Mode de paiement avec icône et couleur
- Liste des articles commandés avec image, quantité, prix unitaire
- Récapitulatif : sous-total, livraison gratuite, total

3.8 Profil utilisateur

- Avatar avec initiales du nom
- Affichage nom, email, type de cheveux
- Modification du profil (nom, email, type de cheveux)
- Changement de mot de passe sécurisé (vérification ancien mot de passe)

- Menu : Historique commandes, Notifications, Mes adresses, Aide
- Déconnexion avec confirmation

3.9 Notifications

- Liste des notifications avec badge de non-lues
- Types : changement de statut commande, promos, stock disponible, confirmation
- Couleurs et icônes différentes par type
- Tap : marquer une notification comme lue (point doré disparaît)
- Bouton 'Tout lire' dans l'AppBar
- Notifications automatiques créées en BDD quand l'admin change le statut d'une commande

4. Fonctionnalités côté Administrateur

Accessible uniquement avec le compte admin (role = 'admin' en BDD). Interface à 3 onglets.

4.1 Onglet Stats

- Chiffre d'affaires total
- Nombre total de commandes
- Nombre de commandes en attente
- Nombre de clients inscrits
- Aperçu des 3 dernières commandes

4.2 Onglet Commandes

- Liste complète de toutes les commandes
- Affichage : numéro de commande, nom client, type de cheveux, date/heure, statut, total
- Tap : bottom sheet pour changer le statut
- 5 statuts disponibles : En attente / Confirmée / En livraison / Livrée / Annulée
- Changement de statut → notification automatique envoyée au client
- Pull-to-refresh pour actualiser

4.3 Onglet Stocks

- Liste de tous les produits avec image, nom, prix
- Alerte rouge si stock ≤ 10 (badge '⚠ Stock faible')
- Boutons + et - pour modifier le stock en temps réel (sauvegarde BDD)

5. Base de données et diagramme de classes

5.1 Schéma des tables

Table : Utilisateur

Champ	Type	Contrainte	Description
id	INT	PK AUTO_INCREMENT	Identifiant unique
nom	VARCHAR(100)	NOT NULL	Nom complet
email	VARCHAR(150)	UNIQUE NOT NULL	Adresse email
mot_de_passe	VARCHAR(255)	NOT NULL	Hash bcrypt
type_cheveux	VARCHAR(50)		Bouclés / Lisses / etc.
role	ENUM	DEFAULT 'client'	client ou admin
created_at	TIMESTAMP	DEFAULT NOW()	Date création

Table : Produit

Champ	Type	Contrainte	Description
id	INT	PK AUTO_INCREMENT	Identifiant unique
nom	VARCHAR(150)	NOT NULL	Nom du produit
prix	DECIMAL(10,2)	NOT NULL	Prix en FCFA
categorie	VARCHAR(50)		Huiles / Shampoings / Soins
stock	INT	DEFAULT 0	Quantité en stock

Table : Panier

Champ	Type	Contrainte	Description
id	INT	PK AUTO_INCREMENT	Identifiant
utilisateur_id	INT	FK → Utilisateur	Propriétaire du panier
produit_id	INT	FK → Produit	Référence produit
nom	VARCHAR(150)		Nom du produit
prix	DECIMAL(10,2)		Prix unitaire
image	VARCHAR(255)		Chemin image local
quantite	INT	DEFAULT 1	Quantité commandée
created_at	TIMESTAMP	DEFAULT NOW()	Date ajout

Table : Commande

Champ	Type	Contrainte	Description
id	INT	PK AUTO_INCREMENT	Identifiant commande
utilisateur_id	INT	FK → Utilisateur	Client concerné
total	DECIMAL(10,2)	NOT NULL	Montant total FCFA
statut	ENUM	DEFAULT 'en_attente'	en_attente / confirmée / en_livraison / livrée / annulée
nom_livraison	VARCHAR(100)		Nom destinataire
telephone	VARCHAR(20)		Téléphone livraison
adresse	TEXT		Adresse livraison
mode_paiement	VARCHAR(50)		orange_money / mtn_money / wave / livraison / carte
created_at	TIMESTAMP	DEFAULT NOW()	Date commande

Table : CommandeDetail

Champ	Type	Contrainte	Description
id	INT	PK AUTO_INCREMENT	Identifiant
commande_id	INT	FK → commandes	Commande parente
produit_id	INT	FK → produits	Produit commandé
nom_produit	VARCHAR(255)		Nom au moment de l'achat
quantite	INT		Quantité commandée
prix_unitaire	DECIMAL(10,2)		Prix au moment de l'achat

Table : Notification

Champ	Type	Contrainte	Description
id	INT	PK AUTO_INCREMENT	Identifiant
utilisateur_id	INT	FK → Utilisateur	Destinataire
titre	VARCHAR(150)	NOT NULL	Titre de la notification
message	TEXT	NOT NULL	Contenu détaillé
type	VARCHAR(50)		commande / promo / stock
lu	TINYINT	DEFAULT 0	0 = non lu, 1 = lu
created_at	TIMESTAMP	DEFAULT NOW()	Date création

5.2 Diagramme de classes

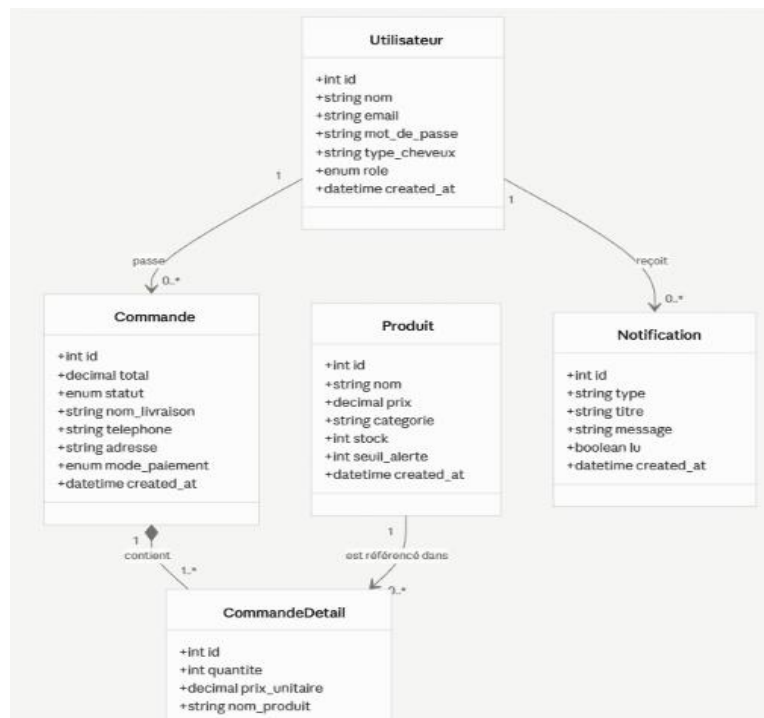


Fig. 1 : Diagramme de classes de l'application AYANA HAIR

Description : Le diagramme de classes ci-dessus présente la structure statique de la base de données de l'application Ayana Hair, composée de cinq entités principales.

La classe Utilisateur est centrale au système : elle représente à la fois les clients et l'administrateur, distingués par l'attribut rôle. Un utilisateur peut passer plusieurs commandes et recevoir plusieurs notifications, ce qui se traduit par des associations de type un-à-plusieurs vers les classes Commande et Notification.

La classe Commande regroupe les informations relatives à chaque achat : le montant total, le statut de traitement (en attente, confirmée, en livraison, livrée), les coordonnées de livraison ainsi que le mode de paiement choisi. Elle entretient une relation de composition avec la classe CommandeDetail, qui détaille chaque ligne de commande c'est-à-dire chaque produit commandé avec sa quantité et son prix unitaire au moment de l'achat.

La classe Produit représente le catalogue disponible à la vente. Elle est liée à CommandeDetail par une association de référence, permettant de retrouver quel produit correspond à chaque ligne de commande tout en conservant un historique des prix indépendant des éventuelles modifications du catalogue.

Enfin, la classe Notification stocke les messages envoyés aux utilisateurs, qu'il s'agisse de confirmations de commande ou d'offres promotionnelles.

6. API REST (pour les routes)

6.1 Authentification

Méthode	Route	Description
POST	/api/inscription	Créer un compte utilisateur
POST	/api/connexion	Connexion et génération du token JWT

6.2 Produits

Méthode	Route	Description
GET	/api/produits	Récupérer tous les produits
PUT	/api/produits/:id/stock	Modifier le stock d'un produit

6.3 Panier

Méthode	Route	Description
POST	/api/panier	Ajouter un article (ou incrémenter quantité)
GET	/api/panier/:utilisateur_id	Récupérer le panier d'un utilisateur
DELETE	/api/panier/:id	Supprimer un article du panier
DELETE	/api/panier/vider/:utilisateur_id	Vider tout le panier

6.4 Commandes

Méthode	Route	Description
POST	/api/commandes	Créer une commande avec ses détails
GET	/api/commandes/:utilisateur_id	Historique des commandes client
GET	/api/commandes/details/:commande_id	Détails d'une commande

6.5 Administration

Méthode	Route	Description
GET	/api/admin/stats	Statistiques globales (CA, commandes, clients)
GET	/api/admin/commandes	Toutes les commandes avec infos client
PUT	/api/admin/commandes/:id/statut	Changer le statut + notifier le client
GET	/api/admin/stocks	Stocks de tous les produits
PUT	/api/admin/stocks/:id	Modifier le stock d'un produit

6.6 Notifications

Méthode	Route	Description
GET	/api/notifications/:utilisateur_id	Récupérer les notifications
PUT	/api/notifications/:id/lu	Marquer une notification comme lue
PUT	/api/notifications/tout-lire/:id	Tout marquer comme lu

7. Sécurité

Mesure	Détail
Hashage mots de passe	bcryptjs avec salt factor 10 (irréversible)
Authentification	JWT signé avec secret, expiration 24h
Stockage local	SharedPreferences chiffré (token + infos user)
Requêtes SQL	Requêtes préparées avec paramètres (protection injection SQL)
CORS	Activé sur le serveur Express pour les requêtes cross-origin
Rôles	Vérification côté serveur du rôle admin vs client

8. Structure des fichiers

8.1 Frontend Flutter

Fichier	Rôle
lib/main.dart	Point d'entrée : ChangeNotifierProvider + MaterialApp
lib/services/api_service.dart	Toutes les méthodes HTTP vers l'API
lib/services/panier_manager.dart	Gestion d'état du panier (Provider)
lib/screens/splash_screen.dart	Écran de chargement initial
lib/screens/login_screen.dart	Connexion utilisateur
lib/screens/register_screen.dart	Inscription utilisateur
lib/screens/home_screen.dart	Accueil avec bannière et grille produits
lib/screens/boutique_screen.dart	Catalogue avec filtres
lib/screens/panier_screen.dart	Panier avec résumé et actions
lib/screens/checkout_screen.dart	Tunnel de commande 3 étapes
lib/screens/historique_screen.dart	Liste des commandes passées
lib/screens/commande_detail_screen.dart	Détails d'une commande
lib/screens/profil_screen.dart	Profil et paramètres utilisateur
lib/screens/notification_screen.dart	Centre de notifications
lib/screens/admin_screen.dart	Dashboard administrateur

8.2 Backend Node.js

Fichier	Rôle
server.js	Point d'entrée : toutes les routes API REST
package.json	Dépendances : express, mysql2, bcryptjs, jsonwebtoken, cors

9. Technologies utilisées

Technologie	Usage
Flutter	Framework UI multi-plateforme
Dart	Langage de programmation Flutter
Provider	Gestion d'état (PanierManager)
http	Requêtes HTTP vers l'API
shared_preferences	Stockage local token/user
Node.js	Serveur backend JavaScript
Express.js	Framework API REST
MySQL	Base de données relationnelle
bcryptjs	Hashage des mots de passe
jsonwebtoken	Génération et validation JWT

10. Livrables

Livrable	Description
Code source Flutter	Projet complet lib/ avec tous les écrans et services
Code source Backend	server.js + package.json
Base de données	Scripts SQL de création des tables
Assets	Images produits dans assets/images/
Cahier des charges	Ce document
Documentation API	Routes, paramètres et réponses dans ce document

11. Galerie (captures d'écran des interfaces de l'application sur émulateur Android Google Pixel 7)

- Écran côté Administrateur

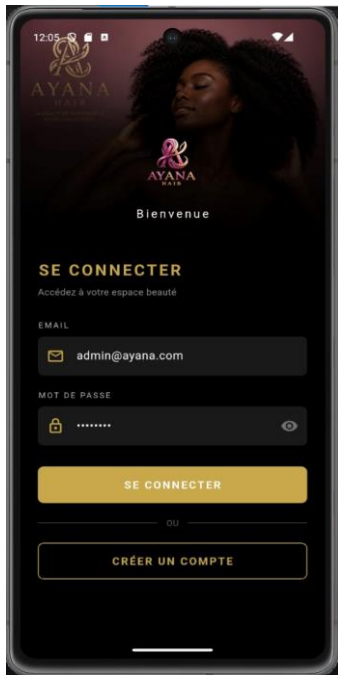


Fig. 2 : Écran de connexion

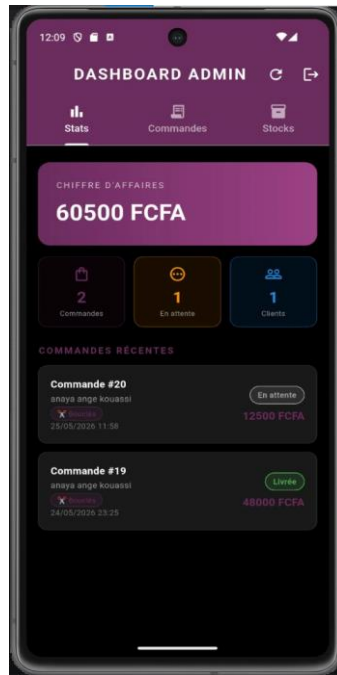


Fig. 3 : Dashboard Admin (Stats)

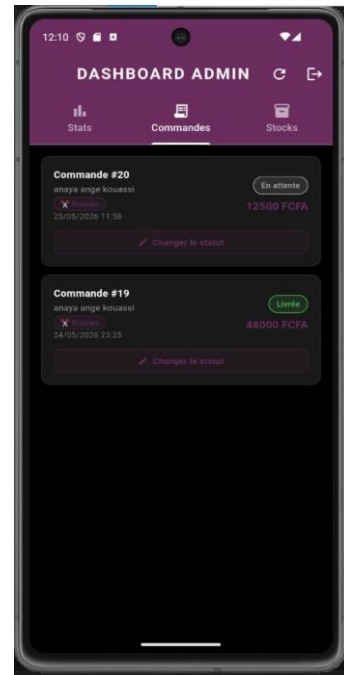


Fig. 4 : Dashboard Admin (Commandes)

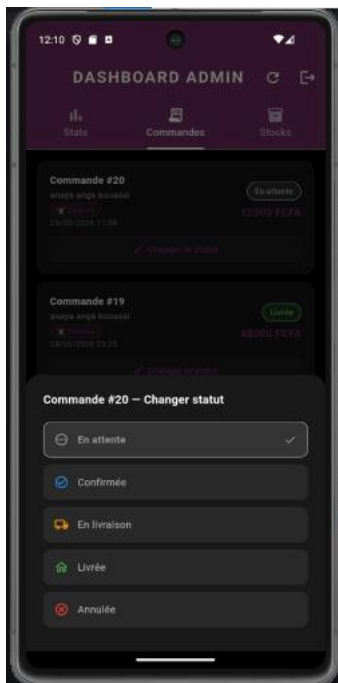


Fig. 5 : Changer le statut d'une commande

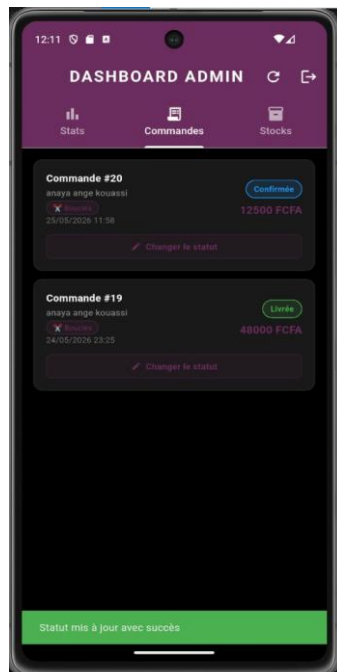


Fig. 6 : Statut de la commande mis à jour (Confirmée)

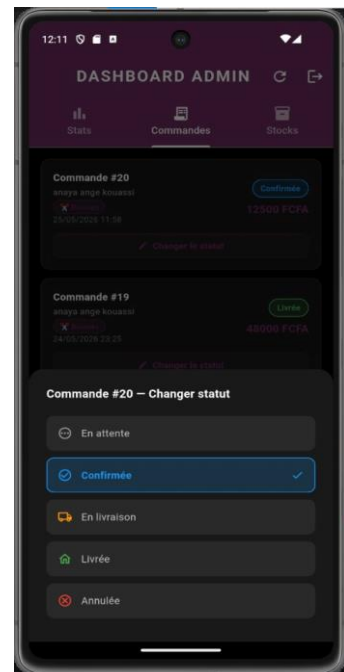


Fig. 7 : Bottom avec statut actif de la commande

AYANA HAIR

Cahier des Charges Technique

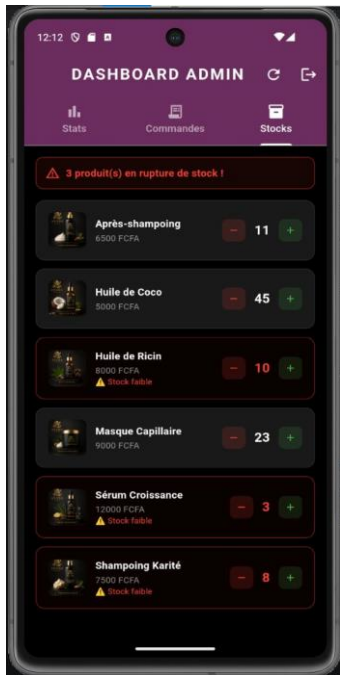


Fig. 8 : Dashboard Admin (Stocks)

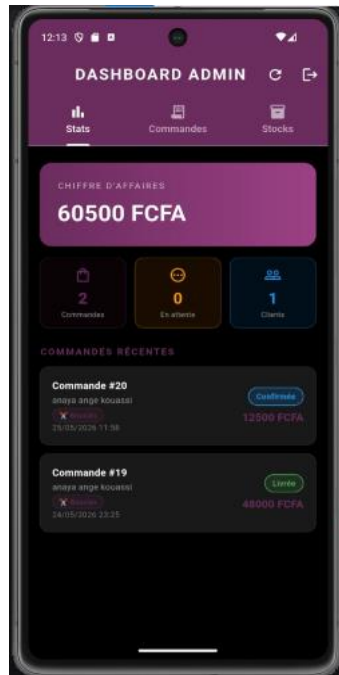


Fig. 9 : Stats après mise à jour



Fig. 10 : Dialog de déconnexion

• Écran côté utilisateur

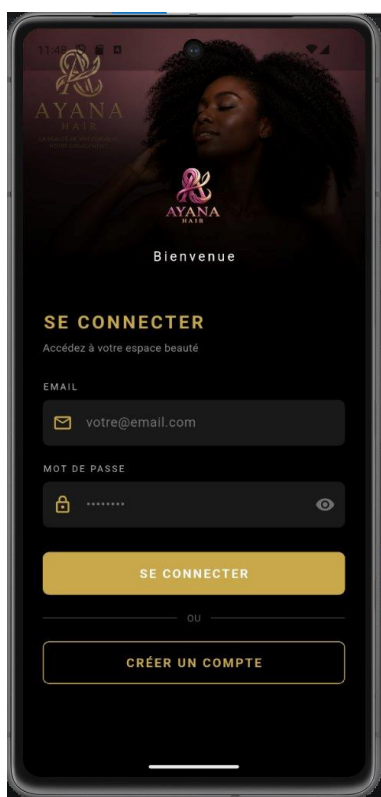


Fig. 11 : Écran de connexion (Se connecter)

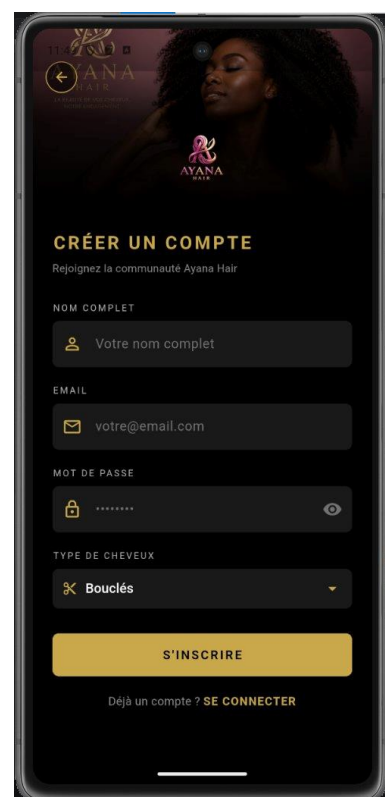


Fig. 12 : Écran de création de compte

AYANA HAIR

Cahier des Charges Technique

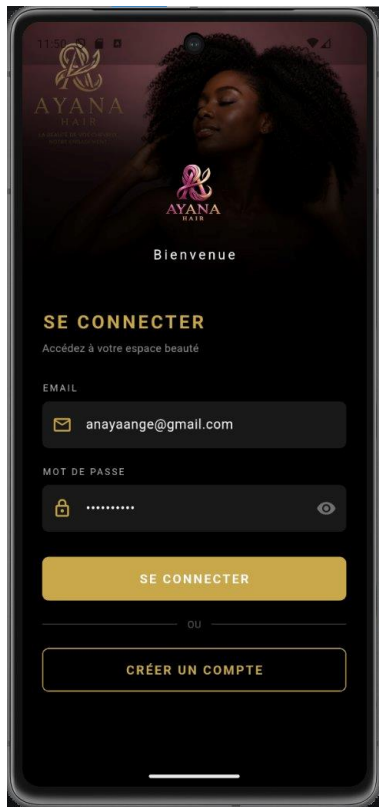


Fig. 13 : Connexion avec identifiants remplis

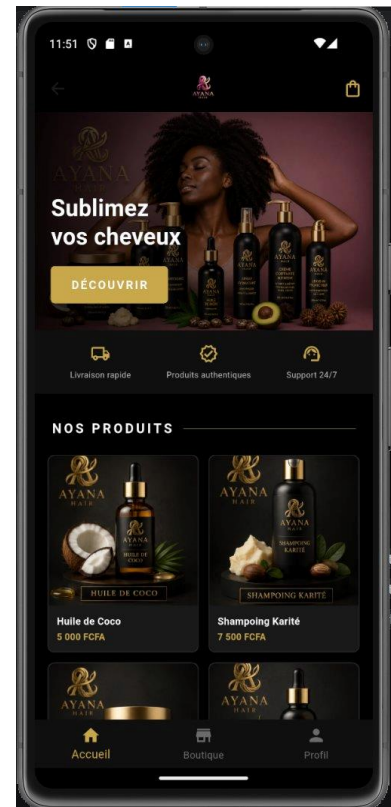


Fig. 14 : Page d'accueil (page principale)

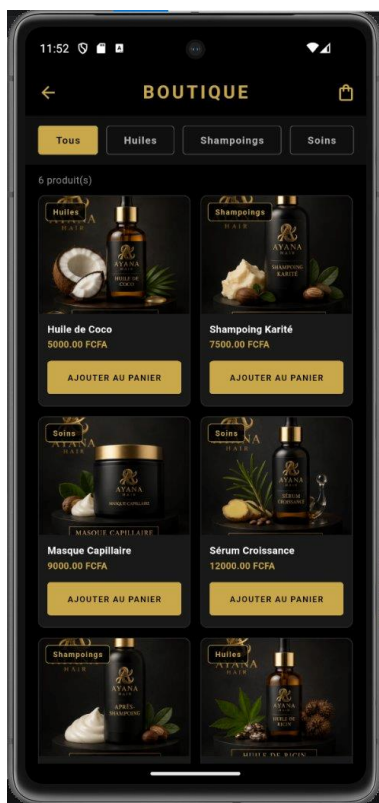


Fig. 15 : Page Boutique avec vue de tous les produits

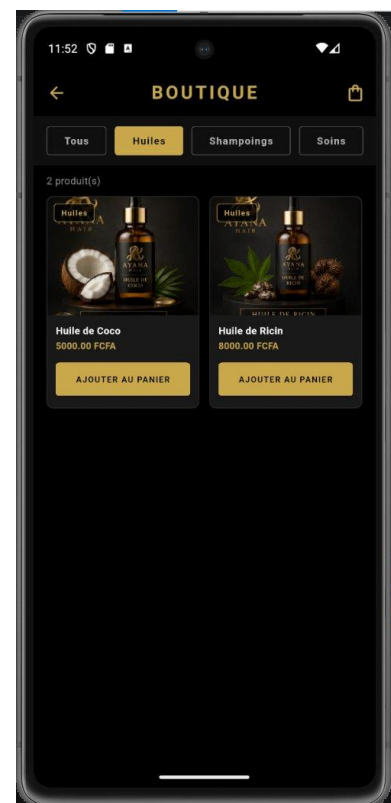


Fig. 16 : Page Boutique (filtre Huiles)

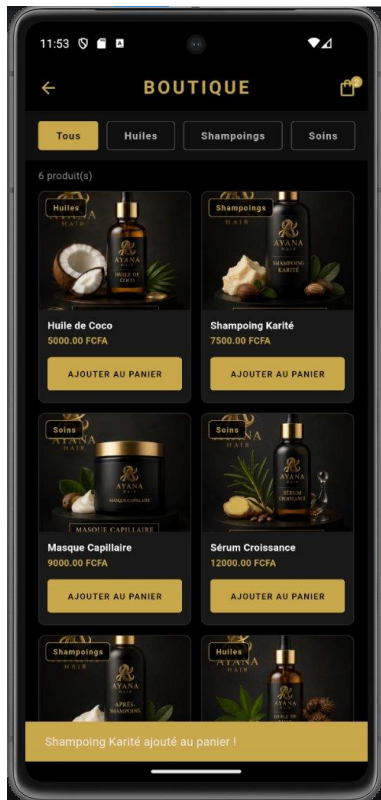


Fig. 17 : Boutique ajout au panier avec notification de confirmation

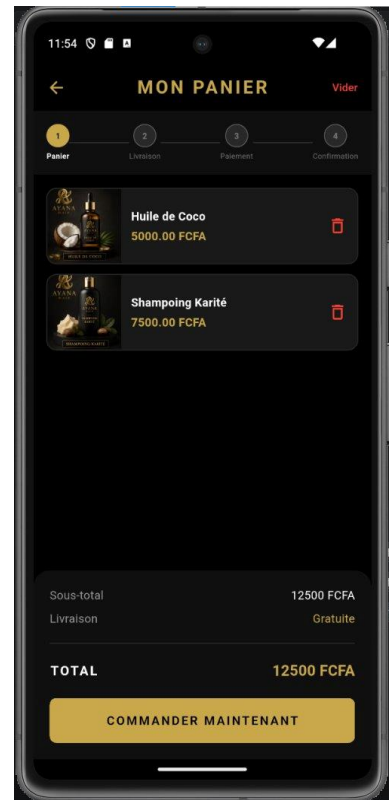


Fig. 18 : Mon Panier

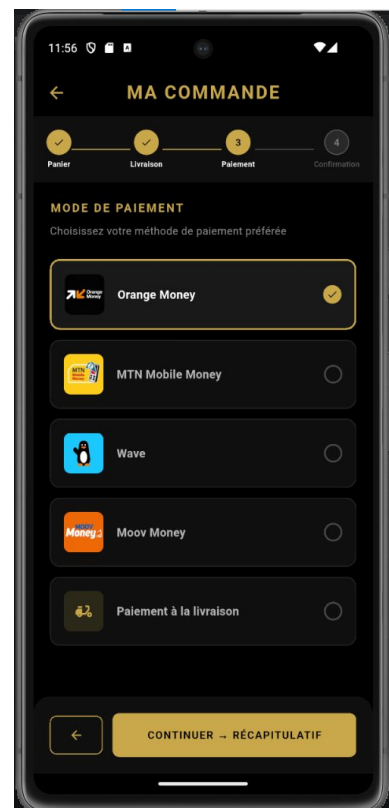
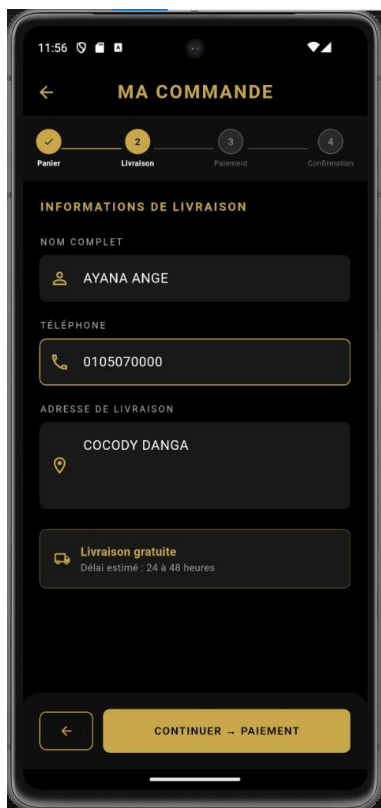


Fig. 19 :Ma Commande (Informations delivraison) **Fig. 20 :** Ma Commande (choix du mode de paiement)

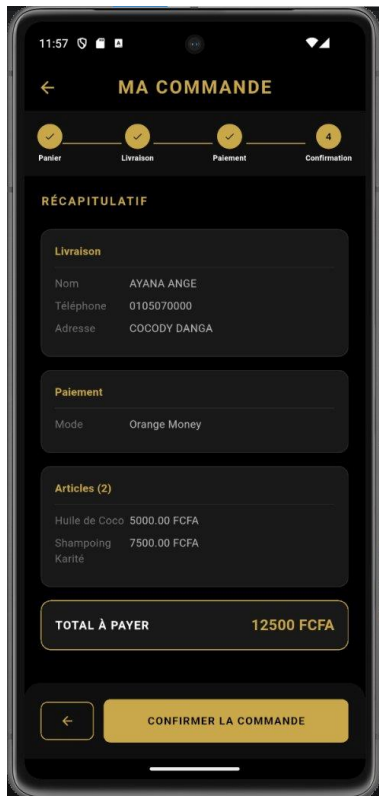


Fig. 21 : Ma Commande (Récapitulatif)

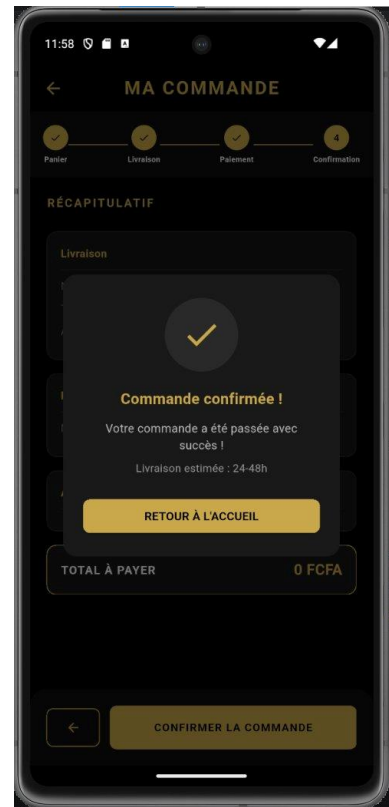


Fig. 22 : Commande confirmée



Fig. 23 : Panier vide

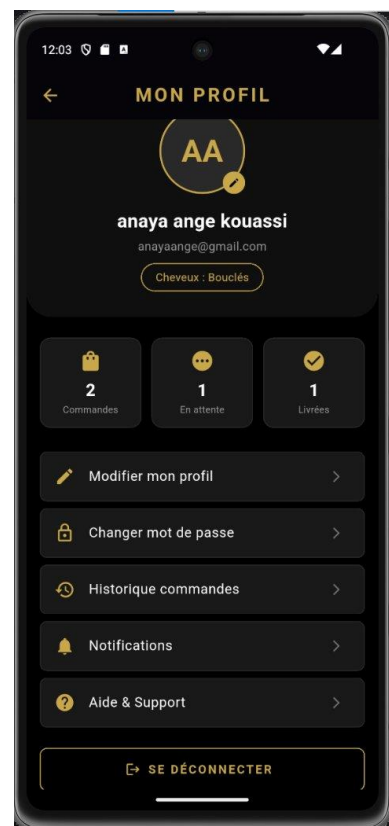


Fig. 24 : Mon Profil

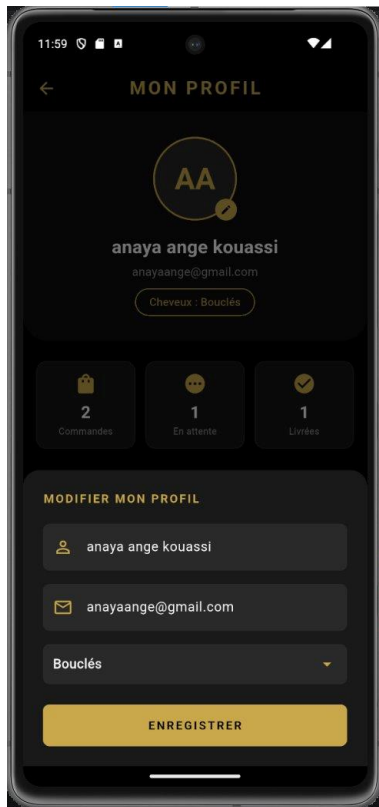


Fig. 25 : Modifier mon profil

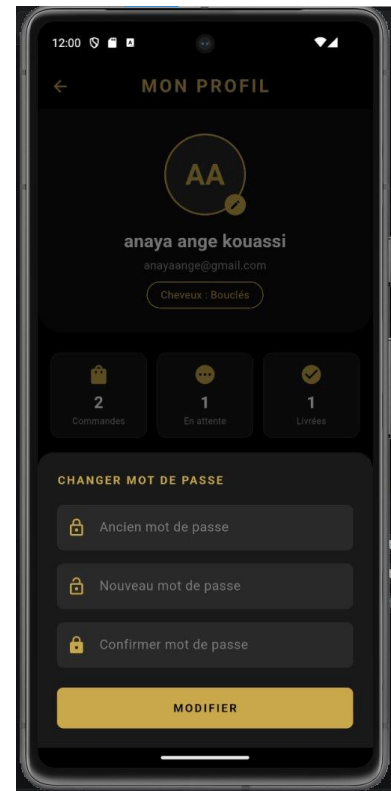


Fig. 26 : Changer mot de passe

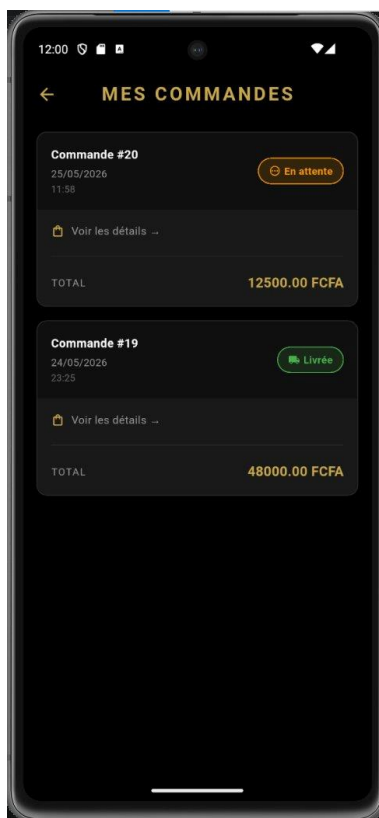


Fig. 27 : Mes Commandes

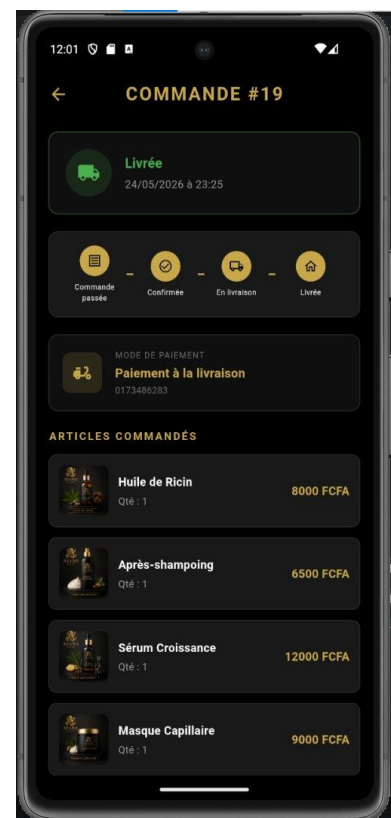


Fig. 28 : Détail commande livrée (#19)

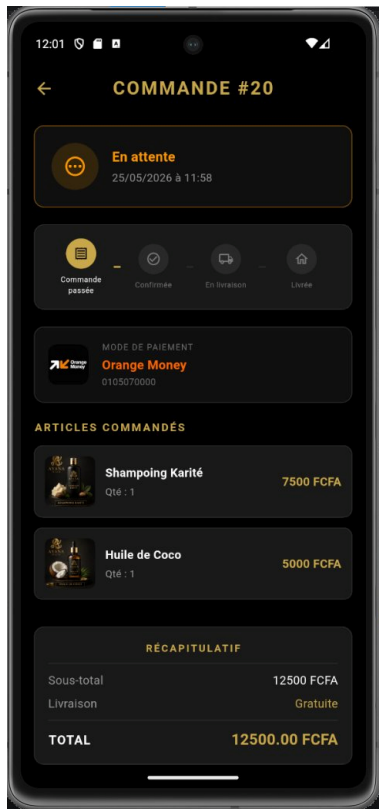


Fig. 29 : Détail commande en attente (#20)

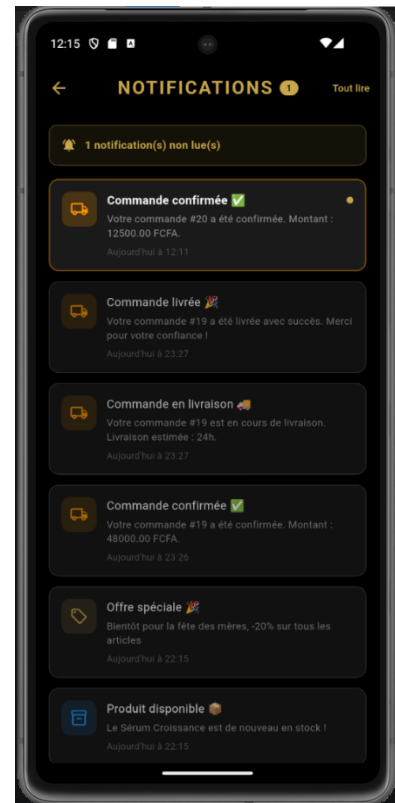


Fig. 30 : Notifications (après changement du statut de la commande par l'administrateur ou information promotionnelle)